Schenider App - One-Page Repository Summary

Repo scanned: /Users/ali/Scheniders

What It Is

A Next.js order management web app for the Schenider team, with Supabase as the backend and a dashboard-style UI. The repository shows operations tooling for orders, analytics, printing, customers/products, and team workflows.

Who It Is For

Primary persona: Schenider business operations staff (admin/employee/viewer roles) managing perfume orders and delivery workflows in Iraq.

What It Does

- Runs a centralized orders workspace with filters, saved views, timeline/notes, duplicate checks, and bulk actions.
- Supports order lifecycle operations including status workflows, cancellation/rejection handling, split orders, and return flow states.
- Provides analytics and finance screens for revenue/profit views, expenses, period comparison, and report export (CSV/Excel/print).
- Includes printing operations for labels/invoices, batch creation/history, reprint warnings, and post-print status updates.
- Offers dedicated modules/pages for customers, products, team analytics, reports, and settings.
- Enforces auth and role-based access control with permission checks and guarded pages/routes.
- Integrates with LightFunnels via webhook ingestion and manual sync APIs, plus scheduled no-response automation.

How It Works (Repo Evidence)

- UI layer: Next.js App Router pages under app/ (dashboard, orders, analytics, printing, products, customers, team, reports, settings).
- Composition layer: app/layout.tsx wraps pages with AuthProvider, QueryProvider, OrdersProvider, PWAProvider, ThemeProvider, LanguageProvider, and DashboardLayout.
- State/data layer: useOrders() uses React Query cache/mutations plus Supabase realtime (postgres_changes on orders) with polling fallback and offline queue handling.
- Backend/data services: Supabase browser/server clients handle auth and database access; auth provider loads role + permission data from role/user permission tables.
- Server integration layer: /api/webhooks/lightfunnels (order ingestion), /api/lightfunnels/sync (token/test/sync), /api/cron/auto-no-response (scheduled status automation).
- Data flow (compact): Browser UI -> Providers/Hooks -> Supabase client + internal API routes -> Supabase tables/RPC -> realtime/persisted cache updates UI.

How To Run (Minimal)

- Install dependencies: `npm install`
- Create env file: `.env.local` with `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` (see `env.example`).
- Database bootstrap command/script: **Not found in repo.** (README references `supabase-schema.sql`, but that file is not present at repo root.)
- Start dev server: `npm run dev` then open `http://localhost:3000`.

Evidence sampled from: README.md, package.json, app/layout.tsx, hooks/use-orders.ts, components/providers/*, lib/supabase*.ts, middleware.ts, and app/api/* routes.